

NEWS SUMMARIZER & CONNECTOR



Obsidian & the Agent Stack

How we turn a spreadsheet of 8,926 news articles into a living, navigable knowledge graph — and how Claude Code orchestrates the agents that build it.

■ Data & scripts ■ Agent stack ■ Obsidian vault

● What is Obsidian?

A local-first knowledge base that stores everything as plain Markdown files on your own disk — no database, no lock-in, no cloud required.

Plain Markdown

Every note is a `.md` file in a folder ("the vault"). Readable and editable with or without Obsidian.

Wikilinks

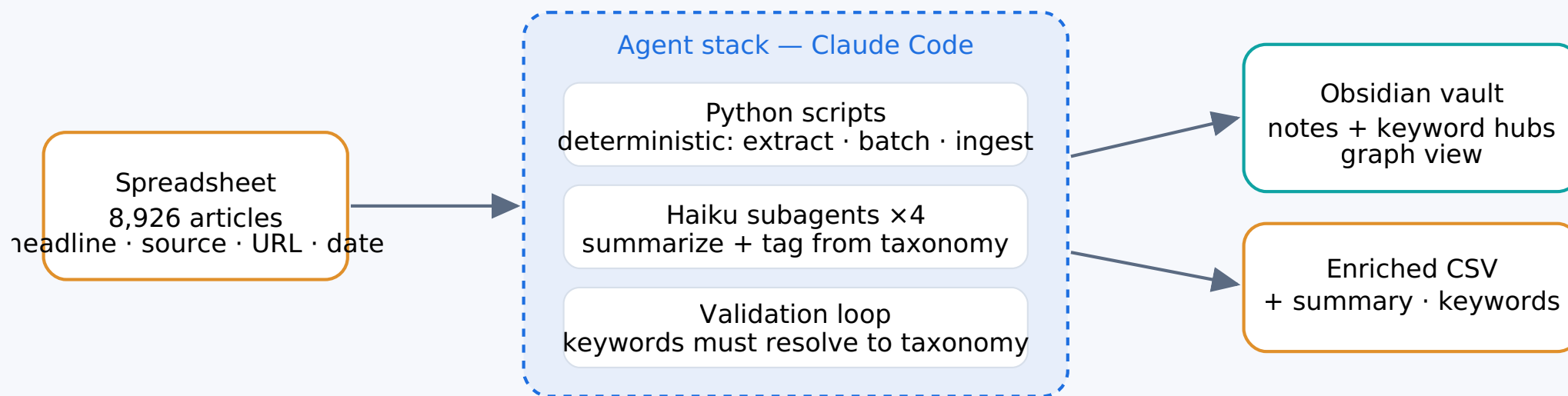
Type `[[Canada]]` and notes link to each other. Links are bidirectional — every note shows its **backlinks**.

Graph view

Obsidian renders all notes and links as an interactive graph. Densely-linked topics cluster together visually.

Why it fits this project: a news corpus is really a web of *themes*. Markdown + wikilinks + the graph turn 8,926 flat rows into something you can see and *walk*.

Spreadsheet → Agents → Knowledge graph



Deterministic plumbing in Python; judgement (summaries & tags) delegated to fast Haiku subagents; everything written as Markdown Obsidian reads natively. **Runs entirely on a Claude subscription — no API key.**

● Why Obsidian, not a database?

For a corpus meant to outlive any single tool, the system of record is plain text on disk — not rows in a service we rent.

Local-first Markdown

Every note is a `.md` file with YAML frontmatter. No server, no schema migrations, no cloud dependency — readable in any editor, today and in ten years.

Git is the sync layer

Because notes are files, version control *is* the database. The agent commits & pushes; you `git pull` and open the vault. Full history, diffable, conflict-aware.

Zero lock-in

Obsidian is a *viewer* over the files, not the system of record. Uninstall it tomorrow and the knowledge graph still exists as plain text the org owns.

Director's lens: the durable asset is the vault — versioned plain text — with **no runtime, no hosting bill, and no vendor between us and our data.**

● Anatomy of the vault

Two kinds of note, linked by wikilinks:

- **Article notes** — one per article, in `vault/articles/news/`.
YAML frontmatter (id, source, url, keywords, entities) + a 1–2 sentence **summary**, a one-line **key takeaway**, and a row of `[[Keyword]]` links.
- **Keyword hubs** — one per taxonomy term, in `vault/keywords/`.
Every article that cites the keyword shows up as a **backlink**, so the hub becomes a theme dashboard for free.

62 keyword hubs · 100 article notes in the pilot · both grow as the pipeline runs.

```
vault/
├── Home.md           entry point + graph tips
├── articles/news/
│   ├── 2026-06-08 ... (n-96a1e9a4).md
│   ├── 2026-06-08 ... (n-23eb2e47).md
│   └── ...           100 article notes
└── keywords/
    ├── AI Adoption.md
    ├── Data Centres.md
    ├── Canada.md
    └── ...           62 keyword hubs
```

● Inside an article note

One Markdown file per article — frontmatter Obsidian parses natively, then three fixed sections:

- `id` — stable `n-<sha1(url)[:12]>`; dedups re-imports.
- `headline · source · url · date` — provenance, verbatim.
- `content_basis` — `full-text` or `headline-only`.
- `keywords` (1–5, canonical) · `entities` (0–4).
- **## Summary** ≤400 · **## Key Takeaway** ≤300 · **## Keywords**
→ `[[wikilinks]]`.

Filename: `YYYY-MM-DD Headline (id).md` — chronological & unique.

```
---
id: n-0b11b0e5d85b
type: news-article
headline: "Cohere signs exploratory AI
  agreement with Québec government"
source: "Beta Kit"
date: 2026-06-09
content_basis: headline-only
keywords: ["Canada", "AI Adoption",
  "AI Policy & Regulation"]
entities: ["Cohere"]
---
```

Summary

Cohere enters an exploratory AI agreement with the Québec government.

Key Takeaway

Signals a push to position Canada as an AI hub.

Keywords

`[[Canada]] · [[AI Adoption]] · [[AI Policy & Regulation]]`

● Keyword hubs become dashboards — for free

One hub note per taxonomy term, in `vault/keywords/`:

- `ensure_hub_notes()` writes the hub the **first time** any article cites the keyword — idempotent; it skips one that already exists.
- All we author is frontmatter `type: keyword` + the one-line taxonomy definition.
- Every note that links `[[Data Centres]]` then appears in that hub's **backlinks** automatically — **Obsidian builds the dashboard; we write nothing per-article into the hub.**
- In the graph, a hub's size scales with its backlink count, so the busiest themes are visually obvious.

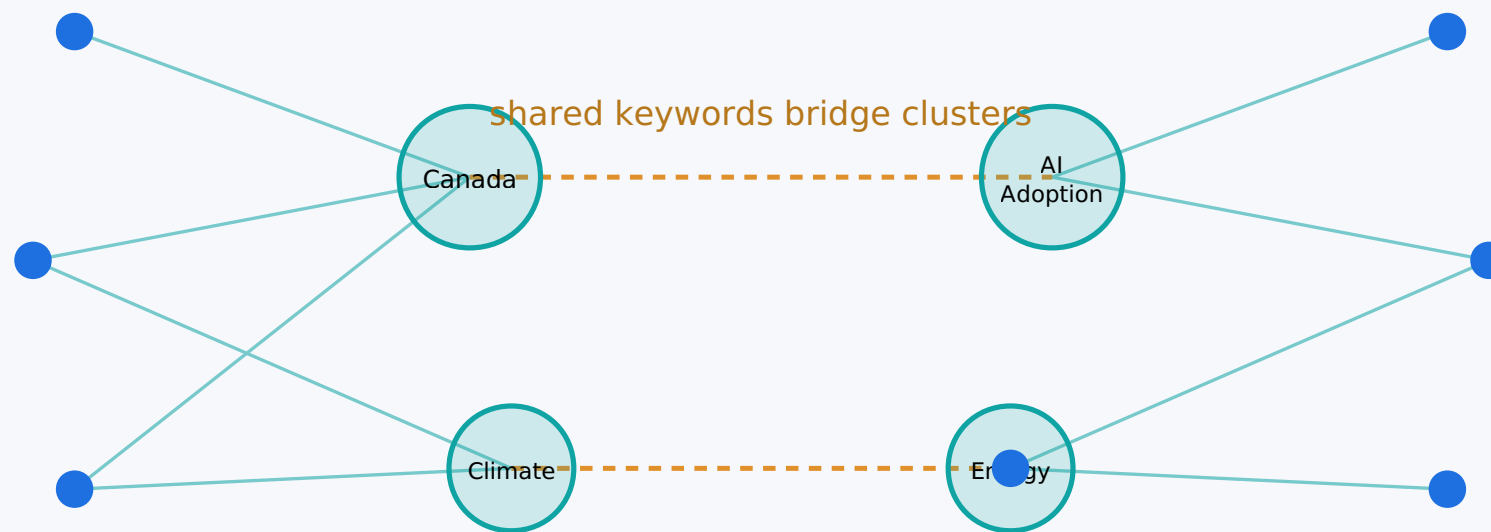
```
# vault/keywords/Data Centres.md
---
type: keyword
---

# Data Centres
Data-centre construction, siting,
energy use

▼ Backlinks (7) — built by Obsidian
· Sobering numbers on data centres'...
· The business of AI's 4 harsh...
· Power-hungry AI is reshaping the grid
· ...
```

● Wikilinks weave the graph

article notes



■ Article note ■ Keyword hub ■ Bridge between themes

● Two ways to see the same graph

Obsidian graph view — native

Nodes are notes, edges are `[[wikilinks]]`. Colour groups split folders (`path:articles/news` vs `path:keywords`); a local graph per note; live filter & zoom. **Zero code — it's just the links.**

`output/graph.html` — standalone

`render_graph.py` reads the ledger and emits a self-contained force-directed canvas — **86 KB, no Obsidian, no server.** Keyword nodes gold & sized by article count; article nodes blue; teal edges; search + click-for-details.

Same structure, two audiences: the article → keyword graph serves Obsidian users *and* anyone with a browser — so the corpus can be shared without installing anything.

● Who does what

Claude Code — orchestrator

Runs on your subscription. Drives the skills, spawns subagents, parses results, handles retries. The conductor, not the labourer.

Skills — the playbooks

`/build-taxonomy` freezes the keyword vocabulary; `/process-articles N` runs the batch pipeline. Repeatable, resumable.

Python scripts — determinism

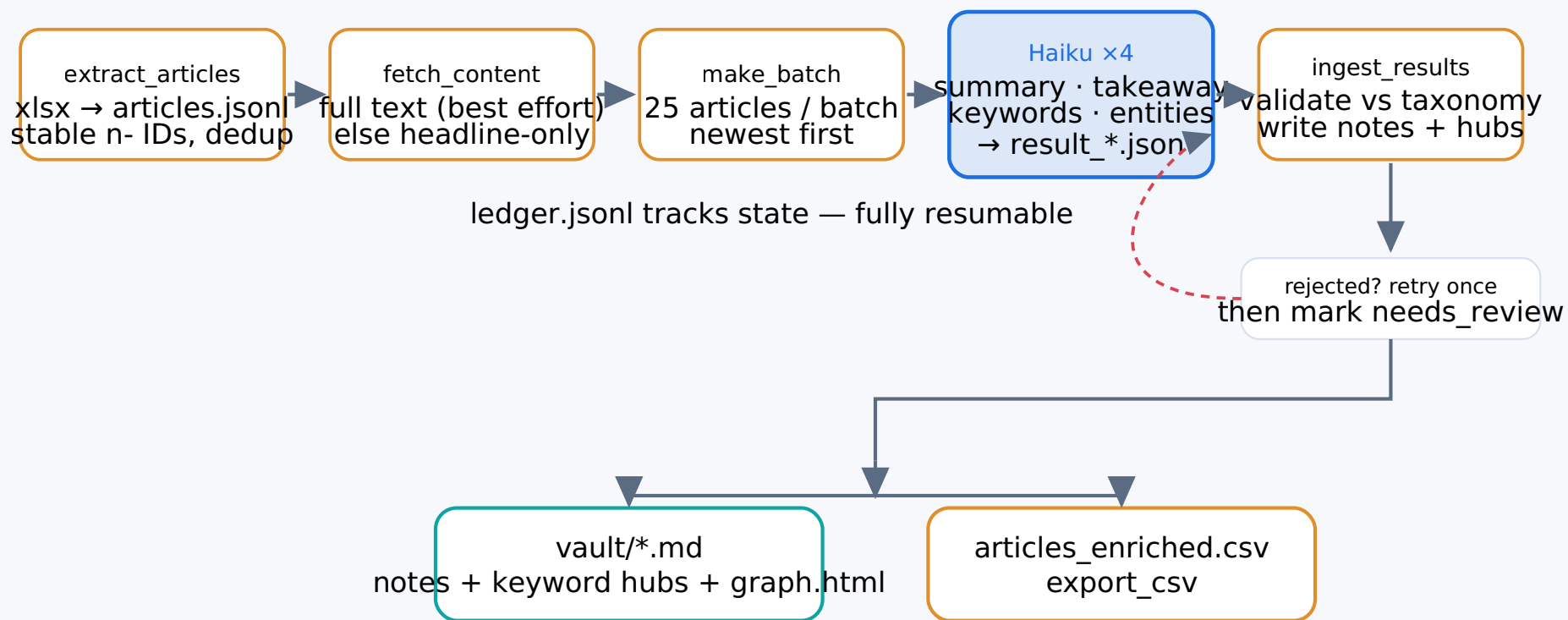
Stdlib only. Extraction, stable IDs, batching, validation, note + CSV writing, graph rendering. No LLM guesswork where rules suffice.

Haiku subagents — judgement

Up to 4 in parallel. Each reads a 25-article batch and returns summary, takeaway, keywords, entities. Cheap, fast, disposable.

Division of labour: rules-based work stays in Python (reproducible, free); only language understanding goes to the model — and to the *cheapest* model that does the job well.

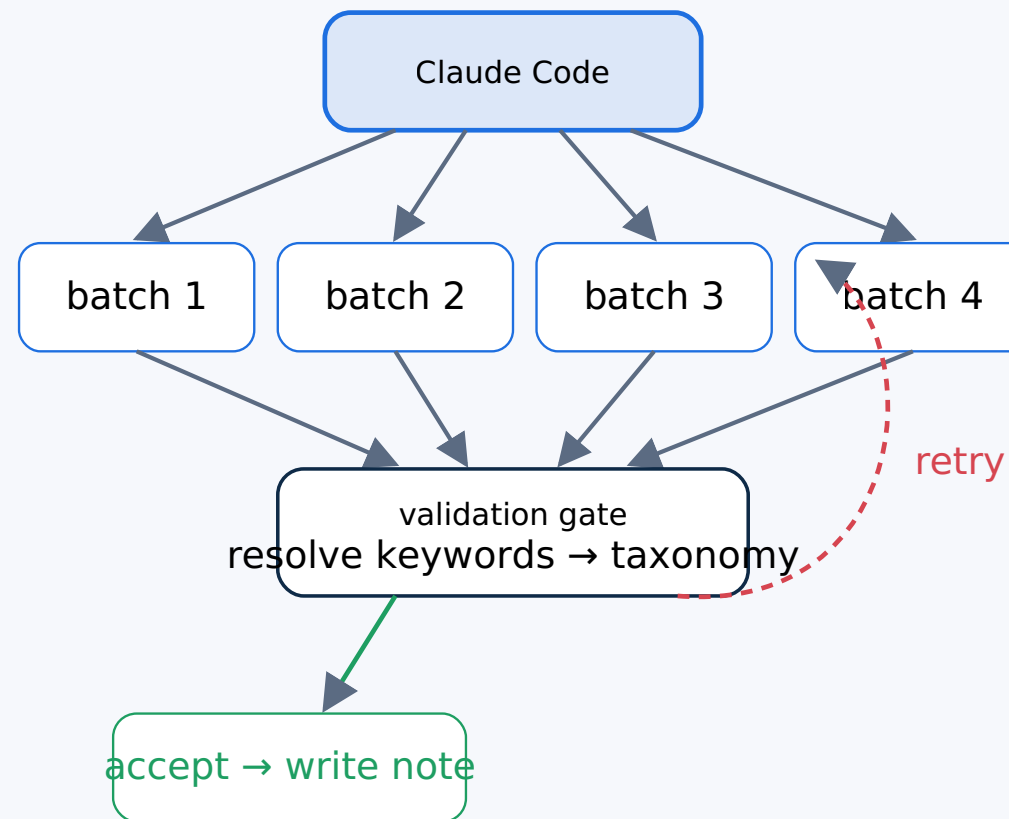
The processing pipeline, end to end



Fan-out & the validation gate

The slow part — reading and summarizing — is parallelised:

- The orchestrator splits pending work into **25-article batches** and launches up to **4 Haiku subagents at once**.
- Each subagent must pick keywords **only from the frozen taxonomy**. Free-text tags are not allowed.
- `ingest_results.py` is the **gate**: keywords are resolved case-insensitively and alias-aware against the taxonomy. Anything that doesn't resolve is rejected.
- Rejected articles get **one retry** (with the reason fed back); a second failure is flagged `needs_review` — **the pipeline never wedges**.

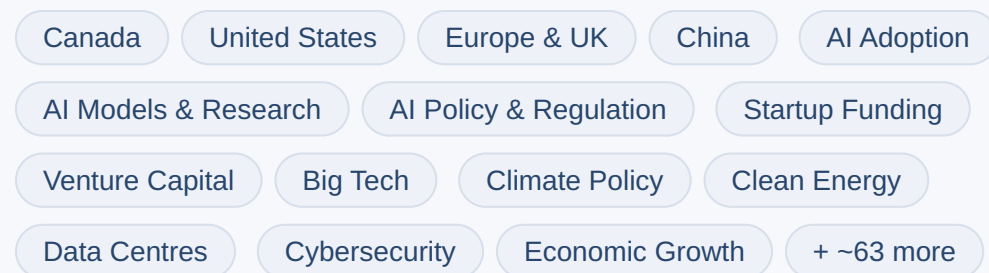


● The controlled taxonomy

78 frozen keywords (62 in use today) are the **shared vocabulary** that makes the graph cohere — and the bridge to the future insight-articles list.

- **Why controlled?** If every article invented its own tags, nothing would cluster. A fixed list of ~78 terms means thousands of articles snap onto the *same* hubs.
- **How built:** `/build-taxonomy` samples 400 stratified headlines, proposes grouped keywords for your review, then `finalize_taxonomy.py` freezes `taxonomy.json` (versioned).
- **Aliases** catch near-misses — `US / USA / America` all resolve to **United States**.

A slice of the vocabulary:



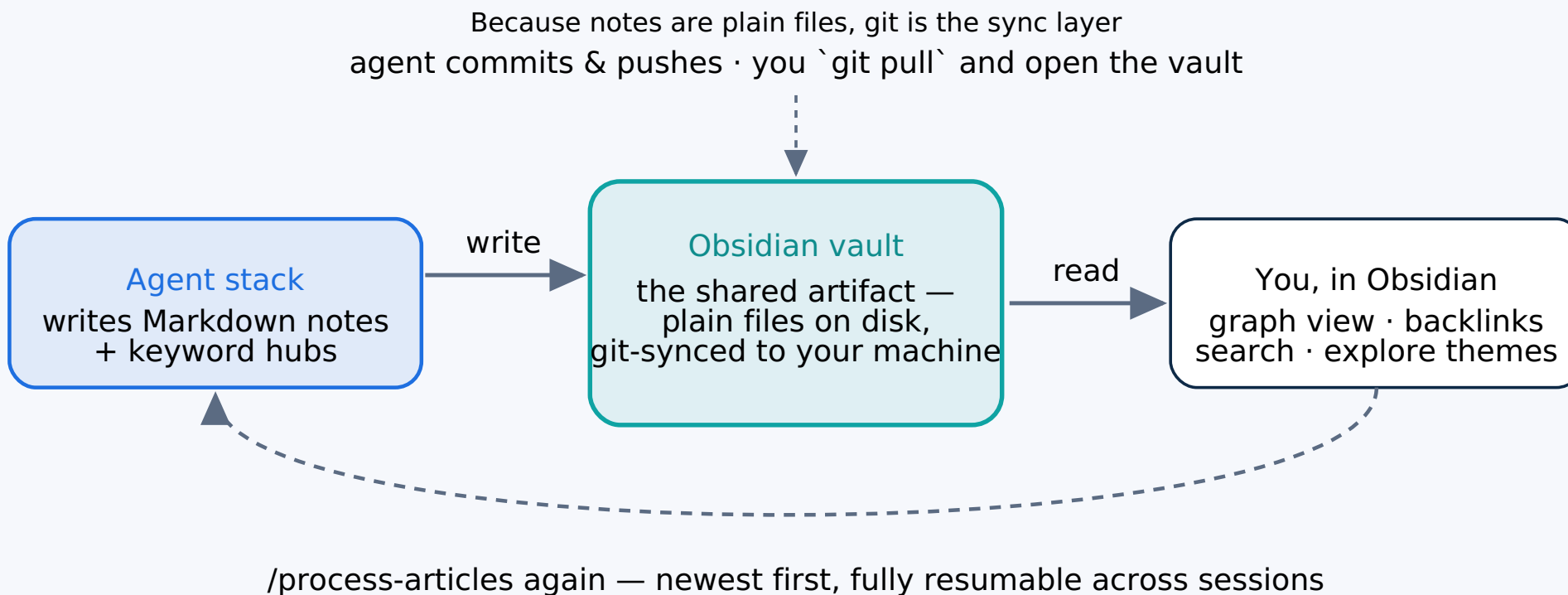
● Obsidian threads through every stage

Obsidian isn't a final export step — its primitives are the target the whole pipeline writes toward.

- `extract_articles` → the stable `id` becomes the note's frontmatter `id` and its filename suffix.
- **Haiku tagging** → each chosen keyword becomes a `[[wikilink]]` in the note body.
- `ingest_results` → writes `vault/articles/news/*.md` and ensures the `vault/keywords/*.md` hubs exist.
- **Obsidian** → backlinks & the graph view *assemble themselves* from those links — no extra build step.
- `render_graph` → mirrors the very same graph to `output/graph.html` for non-Obsidian viewers.

Every stage either produces a vault primitive or reads one — Obsidian is the backbone, not the afterthought.

How the agent stack & Obsidian work together



Current state & next phase

8,926

articles in corpus

100

processed in pilot

62

keyword hubs live

Built today

Extraction with stable dedup IDs · frozen 78-keyword taxonomy · Haiku batch summarization + tagging · Obsidian vault with graph · enriched CSV · standalone `graph.html`. Resumable via `ledger.jsonl`.

Designed, next

Ingest the **insight-articles** list (`i-` IDs) tagged from the *same* taxonomy, then `find_connectors.py` ranks keywords by **(news × insight)** backlinks to surface the bridges between the two maps.

RECAP

A spreadsheet becomes a map you can walk

Scripts do the rules

Deterministic, free, reproducible.

Haiku does the reading

Parallel, validated, cheap.

Obsidian shows the shape

Local Markdown, graph, backlinks.

Open `vault/` as an Obsidian vault · view `output/graph.html` in any browser ·
run `/process-articles 100` to grow it.